Restricted grammar and static analysis as a solution for system programming languages: A language technical report

Fot Nikolai

April 2026

Abstract

This report summarizes the documented design of the Cordell Programming Language (CPL) and the Cordell Compiler, its restricted grammar and advanatages that are reached. The report covers the language **core**, preprocessing, tokenization and markup, **AST** and **IR** generation, CFG and SSA construction, DAG-based optimization, liveness analysis, register allocation, copy propagation, IR lowering, peephole optimization, code generation, static diagnostics, debugging, and the documented language benchmark values against GCC 14.2.0 and Apple Clang 12.0.0.

1 Introduction

CPL is documented as a small systems language and compiler project intended mainly for learning compiler architecture, experimenting with optimizations, and producing low-level code. The project is also connected with the development of operating systems, IoT-applications, etc.

The project is not presented as a feature-rich general-purpose language. The documentation explicitly emphasizes low-level behavior, C-like syntax, assembly affinity, and a restricted language core. The language omits structures, classes, enums, and unions. This restriction is not accidental: it keeps the parser and language surface small so that the compiler pipeline, optimizations, and backend experiments remain the central part of the project.

2 Design Goals

The documentation frames CPL around five practical goals.

- **Simplicity:** the language should stay small and explicit.
- **No safety:** CPL does not attempt to provide memory safety; it prefers low-level control and diagnostics.
- **Systems scope:** intended programs include bootloaders, edge DBMSs, edge file systems, primitive compilers, and small interpreters.
- **Experiments:** the compiler is a test stand for optimizations, symbol-table experiments, perceptron-based heuristics, SMT ideas, and peephole DSLs.
- **Education:** the repository documents a complete compiler-building journey from preprocessing to code generation.

3 Language Core

3.1 Types

CPL is a permissively statically typed language. Values do not freely change type, but the compiler may insert safe widening conversions. Narrowing conversions are never implicit and must be written with `as`.

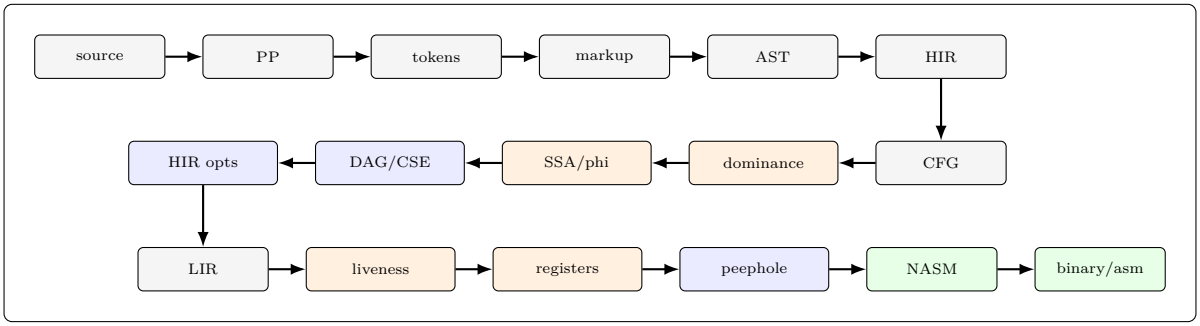

Figure 1: Documentation-level compiler pipeline redrawn in TikZ.

The same explicit cast syntax is also useful when selecting an overloaded function.

Primitive types include signed and unsigned integers from 8 to 64 bits, `f32`, `f64`, and `i0`. The `i0` type is a void-like type for function returns and `ptr i0`. The language does not have a separate Boolean type: zero means false and non-zero means true.

Listing 1: Explicit casts and pointer operations.

```
i32 a = 10 as i32;
u8  b = (a + 20 as i32) as u8;
ptr i32 p = ref a;
dref p = 11 as i32;
```

The main non-primitive containers are `str` and `arr`. A stack string behaves similarly to an array of `i8` with a terminating zero byte. Independent string literals are stored in a read-only section and must be referred to with `ref`. Arrays contain primitive element types and are used with explicit indexing and pointer-style operations.

3.2 Control flow and operations

The documented operation set includes arithmetic, comparisons, update operations, bit operations, and logical operations. Logical `&&` and `||` support lazy evaluation, while `@[not_lazy]` forces both sides to be generated.

The control-flow syntax is C-like but intentionally explicit. An `if` condition is followed by a semicolon. `while` is the ordinary conditional loop. `loop` is an endless loop; it must contain `break` or use `@[counter(...)]`. `switch` supports C-style fallthrough, while `@[no_fall]` adds hidden breaks to cases.

Listing 2: Counted loop and non-fallthrough switch.

```
@[counter(10)] loop {
    : repeated ten times :
}

@[no_fall]
@[straight]
switch tag; {
    case 1; { exit 1; }
    default { exit 0; }
}
```

3.3 Functions

Functions are declared with `function`. Prototypes, default arguments, overloading, local functions, function pointers, lambda functions, and variadic arguments are documented. Overloading is restricted: return-type-only overloading is forbidden, overloads are local-only, and candidates are searched in the same scope. Function pointers use `ptr i0`, but no signature information remains available through such a pointer.

Local functions and lambdas do not capture the parent environment. Therefore they are lightweight logic containers, not full closures.

Listing 3: Local function passed as a pointer.

```
function apply(ptr i0 f, i32 a, i32 b) {
    f(a, b);
```

```
}

@[entry("_start")] function main() {
    function add(i32 a, i32 b) {
        return a + b;
    }
    apply(add, 10 as i32, 20 as i32);
    exit 0;
}
```

4 Annotations and Low-Level Mechanisms

Annotations extend the restricted grammar without adding many new constructs. Table 1 summarizes the documented annotations.

Two built-in mechanisms expose low-level behavior: `syscall` and `asm`. A syscall is invoked like a function and takes a platform-dependent argument list. Inline assembly can receive CPL values and refer to them through numbered placeholders. Inline assembly is documented as not optimized by the compiler.

5 Front End

The preprocessor handles `include`, `define`, `undef`, `ifdef`, and `ifndef`. This is necessary because the project uses C-like header organization: headers contain prototypes and guards, while implementation files contain bodies. After preprocessing, `line` directives preserve source-origin information for tokenization.

Tokenization splits the input byte sequence into basic tokens using DFA-like logic. Markup is a second token-level stage that uses scopes and declarations to attach semantic information to tokens. If `i32 a` is declared in a scope, later visible occurrences of `a` can be marked as `i32`.

The parser then builds an AST using an LL approach. AST-level optimizations include condition unrolling and dead-scope elimination, but the documentation treats later IR-level optimizations as the main optimization path.

Table 1: Documented annotation roles.

Annotation	Role
`naked`	disable entry/exit routines
`align`	align variables or arrays
`section`	place functions/globals in sections
`address`	place a function at an address
`entry`	mark entry point
`no_fall`	add hidden breaks in switch
`not_lazy`	generate both logic operands
`straight`	use linear switch search
`counter`	make counted loop
`hot`, `cold`	control branch placement
`register`	bind local variable to register

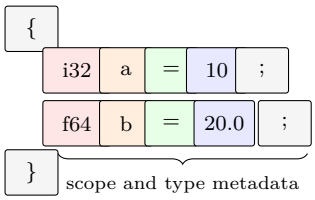

Figure 2: Token markup as a semantic layer over the token stream.

6 HIR, CFG, and SSA

The AST is flattened into HIR, a three-address-code-like representation. This makes later optimization easier. For example, array indexing is expanded into base-address computation, scaled offsets, address addition, and memory store operations.

The CFG is generated from HIR. The documentation describes both one-block-per-command and leader-based splitting. The leader approach starts a block at the first instruction, at jump targets, and immediately after jumps.

SSA form then renames assigned variables so that each assignment creates a unique variable version. Dominators, strict dominators, and dominance frontiers are computed to place phi functions where control-flow paths merge.

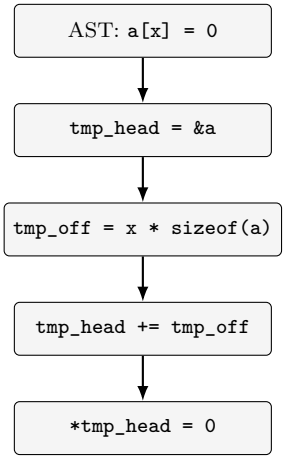

Figure 3: AST-to-HIR flattening into explicit address operations.

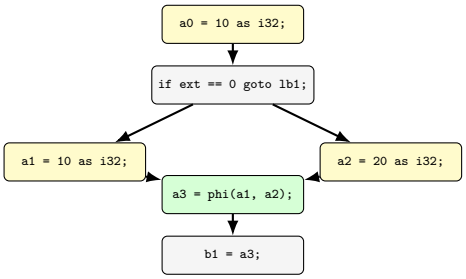

Figure 4: SSA merge point with a phi function.

7 Optimization

After SSA, the compiler can build a DAG that records how values are derived. Nodes with the same hash and compatible dominance can be merged, enabling common subexpression elimination. HIR optimizations also include constant folding and propagation, dead function elimination via a call graph, tail recursion elimination, function inlining, loop canonicalization, and LICM.

Inlining uses a heuristic score. Calls in loops and calls to small functions receive positive score; large callees receive negative score. If the call becomes an inline candidate, the compiler copies the function body, replaces argument loads, and substitutes the return value.

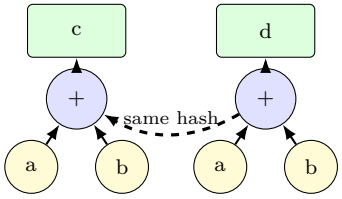

Figure 5: DAG-based common subexpression elimination.

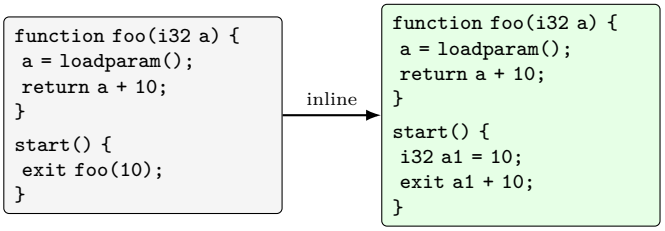

Figure 6: Function inlining by body copying and return substitution.

8 LIR, Liveness, and Registers

LIR is a lower intermediate representation closer to target instructions. It is similar to 3AC but uses a two-address form. It bridges HIR and the target-specific assembly dialect. On x86-64, instruction selection, ABI-related pre-coloring, and register allocation become important at this level.

Liveness analysis computes `USE`, `DEF`, `IN`, and `OUT` for each CFG block:

$$OUT[B] = \bigcup_{S \in succ(B)} IN[S]$$

$$IN[B] = USE[B] \cup (OUT[B] - DEF[B]).$$

The compiler iterates in reverse order until the sets stabilize. If a variable appears in `USE` or `DEF` but not in `OUT`, the compiler can mark it dead with a special `kill` instruction.

For pointers, the compiler uses an alias map. If `b0` owns a reference to `a0`, `a0` must not be killed while

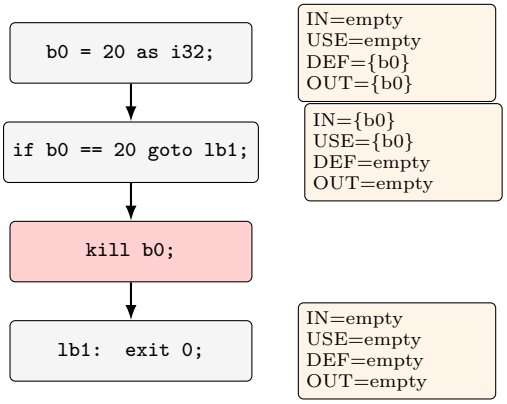

Figure 7: Liveness-driven kill insertion.

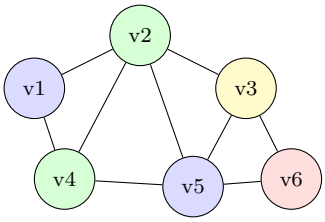

Figure 8: Interference graph coloring for register allocation.

`b0` is live. This is stack-slot reuse and liveness protection, not Rust-style memory safety.

Register allocation builds an interference graph. Vertices are variables; edges connect variables that are live at the same time. Coloring assigns different colors to adjacent nodes, and each color corresponds to a reusable physical register choice.

9 Peephole Optimization and Codegen

The peephole optimizer performs pattern matching and replacement on LIR or assembly-like forms. The simplest documented example is replacing `mov rax, 0` with `xor rax, rax`. The project uses a PTRN DSL to generate pattern matchers instead of coding every pattern by hand.

The documented peephole stage has three passes:

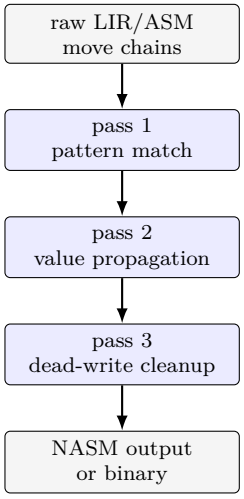

Figure 9: Documented peephole optimization structure.

pattern matching, value propagation through redundant move chains, and dead-write cleanup. Code generation then unwraps special LIR operations such as `EXITOP` and `STRT` and emits NASM assembly.

10 Static Diagnostics and Debugging

The static checker has AST and IR parts. AST warnings cover read-only updates, declarations without initialization, wrong declaration values, missing returns, missing `exit` in `start`, wrong argument counts, wrong argument types, unused function results, return-type mismatches, illegal array access, duplicated branches, reserved function names, dead code, suspicious conversions, inefficient `while 1`, invalid `exit`, targetless `break`, misuse of `i0`, unused expressions, invalid `ref`, and non-even alignment.

IR analysis handles lower-level warnings such as null dereference, null dereference through arguments, constant conditions, propagated constants, wrong casts, and syscall-related checks. Static analysis is off by default and enabled with `-static-analysis`.

For debugging, the compiler can produce assembly with `-emit-asm`. The `-d` flag includes DWARF in-

Table 2: Documented benchmark environment.

Item	Value
OS	MacOS Catalina 10.15.7
CPU	i7 3650QM, quad-core, 2.4 GHz
RAM	16 GB DDR3, 1600 MHz
GCC	GCC 14.2.0, Homebrew
Clang	Apple Clang 12.0.0
CPL	CPL v3.4, MACHO64
Runs	5 per configuration
CPL timing	`py-time`, after compilation
C/C++ timing	`gnu-time`, after compilation

formation, and `lis` is an interrupt point usable with `gdb` or `lldb`. The documentation recommends disabling optimizations when debugging code that must not be transformed.

11 Benchmark Methodology

The benchmark documentation compares GCC 14.2.0, Apple Clang 12.0.0, and CPL v3.4 on MACHO64. GCC and Clang use `-O0` and `-O3`. CPL also uses `-O0` and `-O3`; the documented `-O3` set includes LICM, peephole optimization, CSE, DCE, and constant propagation/folding.

The documentation warns that some C microbenchmarks use `asm volatile` to prevent dead-code elimination, while CPL preserves such loops by default. These tests should therefore be interpreted as loop-overhead measurements rather than broad compiler-performance benchmarks. The documentation also suggests reading CPL times conservatively, with about +30% adjustment for possible measurement issues.

12 Benchmark Results

Lower runtime is better. The following charts reproduce the raw documented values with TikZ bars.

The empty-loop result is best read as loop-overhead evidence because the benchmark is artificial and dead-code elimination behavior differs. Fibonacci reports optimized GCC and optimized CPL at 0.412 seconds, with optimized Clang at 0.417 seconds. String traversal reports optimized CPL slower than optimized Clang and GCC. Brainfuck reports optimized CPL and optimized Clang at 0.403 seconds, with optimized GCC at 0.487 seconds.

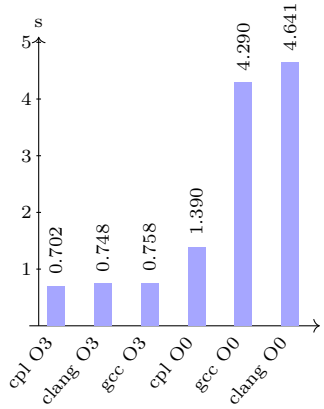

Figure 10: Empty loop benchmark. Optimized CPL assembly: 20 lines, down from 31.

Table 3: Documented CPL assembly-size reductions.

Benchmark	Before	After	Reduction
Empty loop	31	20	35.5%
Fibonacci	48	27	43.8%
String traversal	103	84	18.4%
Brainfuck	769	516	32.9%

13 Limitations

This report is not an independent validation. It does not run the compiler, inspect source code, or reproduce benchmarks. All feature descriptions and measurements come from the uploaded documentation. The diagrams are TikZ reconstructions based on the documentation and reference images.

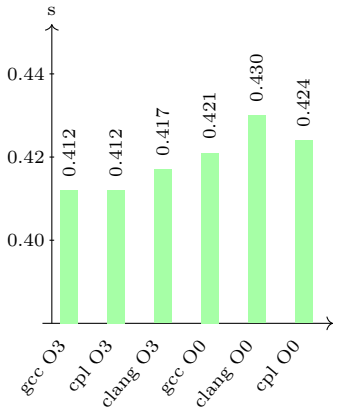

Figure 11: Million Fibonacci benchmark. Optimized CPL assembly: 27 lines, down from 48.

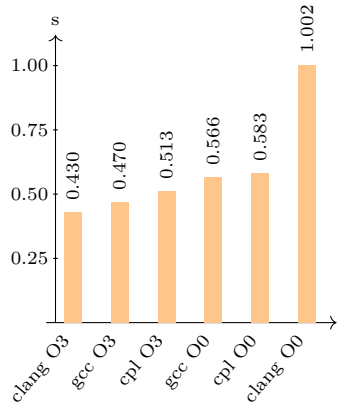

Figure 12: Pointer and string traversal benchmark. Optimized CPL assembly: 84 lines, down from 103.

14 Conclusion

The documentation presents CPL as a compact, explicit, low-level language and Cordell Compiler as an end-to-end compiler laboratory. The pipeline covers preprocessing, tokenization, semantic markup, AST construction, HIR lowering, CFG and SSA, DAG-based optimization, LIR, liveness, register allocation, peephole optimization, and NASM code generation. The benchmark values suggest that the documented optimizer can generate compact and competitive code for selected small examples, while the most important contribution of the project is the coherent educational compiler pipeline.

A Literature Listed in the Documentation

The documentation lists Aarne Ranta's compiler text, the Dragon Book, Appel's *Modern Compiler Implementation in C*, Cytron et al. on SSA and control dependence, Kusswurm's x86 assembly book, and the instruction scheduling text by Jason Robert and Carey Patterson.

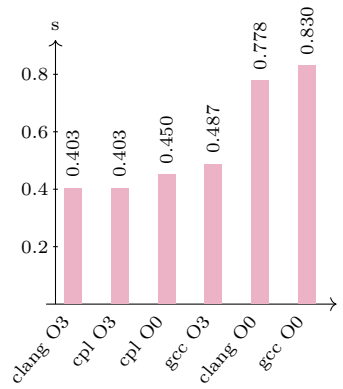

Figure 13: Brainfuck benchmark. Optimized CPL assembly: 516 lines, down from 769.